

# OrbitPay OO

Motor de pagos y suscripciones orientado a objetos

---

## Executive Summary

---

**Autor:** Luis Angel Leyva Castillo

**Versión:** 1.0.0

**Tecnología:** Python 3.12.14

**Licencia:** MIT

**Tipo de entrega:** Proyecto integrador – Módulo 2

Documento ejecutivo de síntesis de la solución, arquitectura, decisiones técnicas, pruebas, calidad y release final de OrbitPay OO.

29 de septiembre de 2026

## 1. Resumen ejecutivo

OrbitPay OO es un motor de pagos y suscripciones desarrollado en Python mediante Programación Orientada a Objetos, diseñado como proyecto integrador del Módulo 2.

Su propósito es transformar un procesamiento monolítico de pagos, caracterizado por lógica condicional anidada, riesgo de cargos duplicados, inconsistencias de saldo y ausencia de pruebas automatizadas, en una solución modular, extensible y verificable.

La solución implementa un modelo de dominio compuesto por cuentas, transacciones y suscripciones, junto con una abstracción polimórfica para métodos de pago. La arquitectura incorpora los patrones Factory, Strategy y Observer, así como principios SOLID y mecanismos de validación, encapsulamiento e idempotencia.

El release final corresponde a la versión **1.0.0**, distribuible como paquete Python mediante archivos `wheel` y `tar.gz`. El proyecto alcanzó **98 % de cobertura de código**, cuenta con pruebas unitarias y de integración, validación estática mediante `mypy -strict`, análisis mediante Ruff, formateo mediante Black y automatización mediante pre-commit y GitHub Actions.

## 2. Problema y objetivo

El escenario inicial de OrbitPay presentaba características propias de un sistema monolítico: lógica de negocio concentrada, decisiones basadas en tipos concretos de métodos de pago, ausencia de separación clara de responsabilidades y falta de pruebas automatizadas.

Estas características dificultaban la incorporación de nuevos métodos de pago y aumentaban el riesgo de errores relacionados con el procesamiento de operaciones y la consistencia del saldo.

El objetivo fue desarrollar una solución orientada a objetos que permitiera:

- encapsular el estado crítico del dominio;
- validar las reglas fundamentales del negocio;
- utilizar polimorfismo para los métodos de pago;
- desacoplar el cálculo de comisiones;
- implementar notificaciones mediante eventos;
- evitar el procesamiento duplicado de transacciones;
- aplicar principios SOLID;
- automatizar pruebas y controles de calidad;
- generar un paquete Python instalable y reproducible.

## 3. Solución y arquitectura

OrbitPay se estructura en cuatro áreas principales:

**Dominio:** contiene `Cuenta`, `Transaccion`, `Suscripcion` y las excepciones del negocio.

**Pagos:** define la abstracción `MetodoPago` y sus implementaciones `Tarjeta`, `Transferencia` y `Wallet`.

**Patrones:** concentra las implementaciones de Factory, Strategy y Observer.

**Motor:** PagoEngine coordina el procesamiento de las operaciones mediante abstracciones y dependencias inyectadas.

El motor no necesita conocer los detalles concretos de cada método de pago. Recibe una instancia de `MetodoPago`, una estrategia de comisión y un gestor de eventos, reduciendo el acoplamiento entre componentes.

#### 4. Diagrama de clases

La Figura 1 presenta la estructura principal de OrbitPay OO y las relaciones entre las entidades del dominio, los métodos de pago, las estrategias de comisión, el motor de pagos y el sistema de eventos.

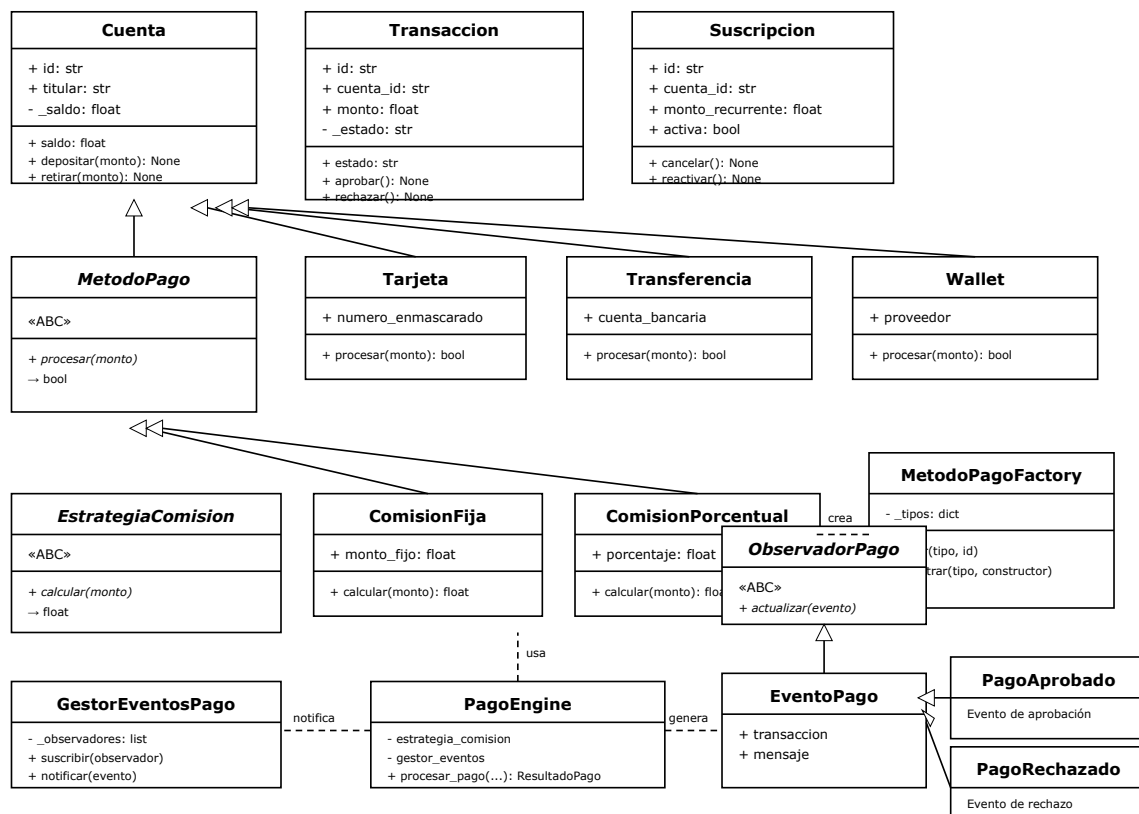


Figura 1: Diagrama de clases de OrbitPay OO.

## 5. Programación Orientada a Objetos

El proyecto evidencia los principales conceptos estudiados durante el módulo.

## 5.1. Encapsulamiento

El saldo de **Cuenta** se mantiene como estado interno y se modifica mediante operaciones controladas como **depositar()** y **retirar()**.

El estado de `Transaccion` también se controla mediante operaciones de dominio como `aprobar()` y `rechazar()`.

## 5.2. Abstracción

`MetodoPago`, `EstrategiaComision` y `ObservadorPago` definen contratos mediante clases abstractas.

## 5.3. Herencia

Los diferentes métodos de pago implementan el contrato común de `MetodoPago`, mientras que las estrategias concretas implementan `EstrategiaComision`.

## 5.4. Polimorfismo

El motor procesa cualquier implementación válida de `MetodoPago` sin utilizar condicionales basados en el tipo concreto.

## 5.5. Composición

`PagoEngine` recibe sus dependencias mediante el constructor, permitiendo sustituir estrategias y gestores de eventos sin modificar el motor principal.

# 6. Patrones de diseño

## 6.1. Factory Method

`MetodoPagoFactory` centraliza la creación de métodos de pago y permite registrar nuevas implementaciones mediante un registro extensible.

## 6.2. Strategy

`EstrategiaComision` separa el algoritmo de cálculo de comisión del motor de pagos.

Se implementaron:

- `ComisionFija`;
- `ComisionPorcentual`.

## 6.3. Observer

`GestorEventosPago` permite notificar eventos de pago sin acoplar directamente el motor con los componentes consumidores.

Se contemplan los eventos:

- `PagoAprobado`;
- `PagoRechazado`.

## 7. Principios SOLID

**SRP – Single Responsibility Principle:** responsabilidades separadas entre dominio, métodos de pago, cálculo de comisiones, eventos y procesamiento.

**OCP – Open/Closed Principle:** Factory y Strategy permiten extender comportamientos sin modificar el motor principal.

**LSP – Liskov Substitution Principle:** las implementaciones de `MetodoPago` cumplen el contrato definido por la abstracción.

**ISP – Interface Segregation Principle:** las interfaces se mantienen pequeñas y enfocadas.

**DIP – Dependency Inversion Principle:** `PagoEngine` depende de abstracciones y recibe sus dependencias mediante inyección.

## 8. Refactorización

Durante el desarrollo se identificaron y corrigieron tres problemas concretos.

### 8.1. Refactor 1 – Estado de transacción

Se substituyó la modificación directa del estado por atributos internos y operaciones de dominio (`aprobar()` y `rechazar()`).

### 8.2. Refactor 2 – Factory extensible

La creación de métodos de pago se transformó en un registro extensible, permitiendo incorporar nuevos tipos sin modificar la lógica central de la Factory.

### 8.3. Refactor 3 – Lógica de rechazo duplicada

La gestión de rechazos se centralizó mediante `_rechazar_pago()`, eliminando duplicación dentro del motor.

## 9. Pruebas y calidad

El proyecto cuenta con pruebas unitarias para dominio, métodos de pago y patrones, además de pruebas de integración para `PagoEngine`.

La ejecución final alcanzó:

**98 % de cobertura**

superando el mínimo requerido de 80 %.

Los controles automatizados incluyen:

- `pytest`;
- `pytest-cov`;
- `Black`;

- Ruff;
- Mypy en modo `strict`;
- pre-commit;
- GitHub Actions.

El pipeline de integración continua ejecuta las verificaciones de formato, análisis estático, tipado y pruebas con un requisito mínimo de 80 % de cobertura.

## 10. Idempotencia y reglas de negocio

El motor mantiene un registro de transacciones procesadas para impedir que una misma transacción sea procesada nuevamente.

También se aplican reglas fundamentales del dominio:

- los montos deben ser mayores que cero;
- el saldo no puede ser negativo;
- una cuenta no puede retirar más fondos de los disponibles;
- los identificadores obligatorios deben ser válidos;
- los estados de las transacciones están restringidos a valores definidos.

## 11. Empaquetado y release

OrbitPay fue empaquetado como versión **1.0.0** mediante `python -m build`.

Se generaron los siguientes artefactos:

- `orbitpay-1.0.0-py3-none-any.whl`;
- `orbitpay-1.0.0.tar.gz`.

La instalación del wheel fue validada en un entorno virtual independiente del entorno de desarrollo.

La prueba confirmó:

- instalación en `site-packages`;
- versión instalada `1.0.0`;
- importación correcta del paquete;
- ejecución funcional de componentes del dominio.

## 12. Resultados

| Indicador              | Resultado                   |
|------------------------|-----------------------------|
| Versión final          | 1.0.0                       |
| Cobertura              | 98 %                        |
| Mínimo requerido       | 80 %                        |
| Patrones implementados | 3                           |
| Refactors documentados | 3                           |
| Tipado                 | Mypy strict                 |
| Formateo               | Black                       |
| Análisis estático      | Ruff                        |
| Automatización         | Pre-commit + GitHub Actions |
| Distribución           | Wheel + Source Distribution |

## 13. Entregables

El repositorio contiene:

- código fuente del dominio;
- métodos de pago;
- patrones de diseño;
- motor de procesamiento;
- pruebas unitarias e integración;
- documentación de las ocho lecciones;
- spikes técnicos;
- diagrama de clases UML;
- README;
- LICENSE;
- configuración de pre-commit;
- workflow de integración continua;
- artefactos de distribución de la versión 1.0.0.

## 14. Conclusión

OrbitPay OO integra los conceptos fundamentales de Programación Orientada a Objetos en una solución de dominio coherente, extensible y verificable.

El proyecto evoluciona desde una problemática monolítica hacia una arquitectura basada en abstracciones, encapsulamiento, polimorfismo, composición y patrones de diseño.

La combinación de pruebas automatizadas, cobertura de código, análisis estático, integración continua y empaquetado reproducible proporciona evidencia técnica del funcionamiento de la solución y de su preparación para distribución como paquete Python.

El release **OrbitPay OO 1.0.0** representa el cierre técnico del proyecto integrador y reúne en un único artefacto las competencias desarrolladas durante el módulo.